



JC4002 Knowledge Representation

Day 10: Answer Set Programming

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 10

Answer Set Programming

1. Introduction
2. Semantics
3. Modeling
4. Variation Logic Programs

Answer Set Programming (ASP) in Nutshell

- ❑ ASP is an approach to **declarative problem solving**, combining
 - a rich yet simple modeling language
 - with high-performance solving capacities

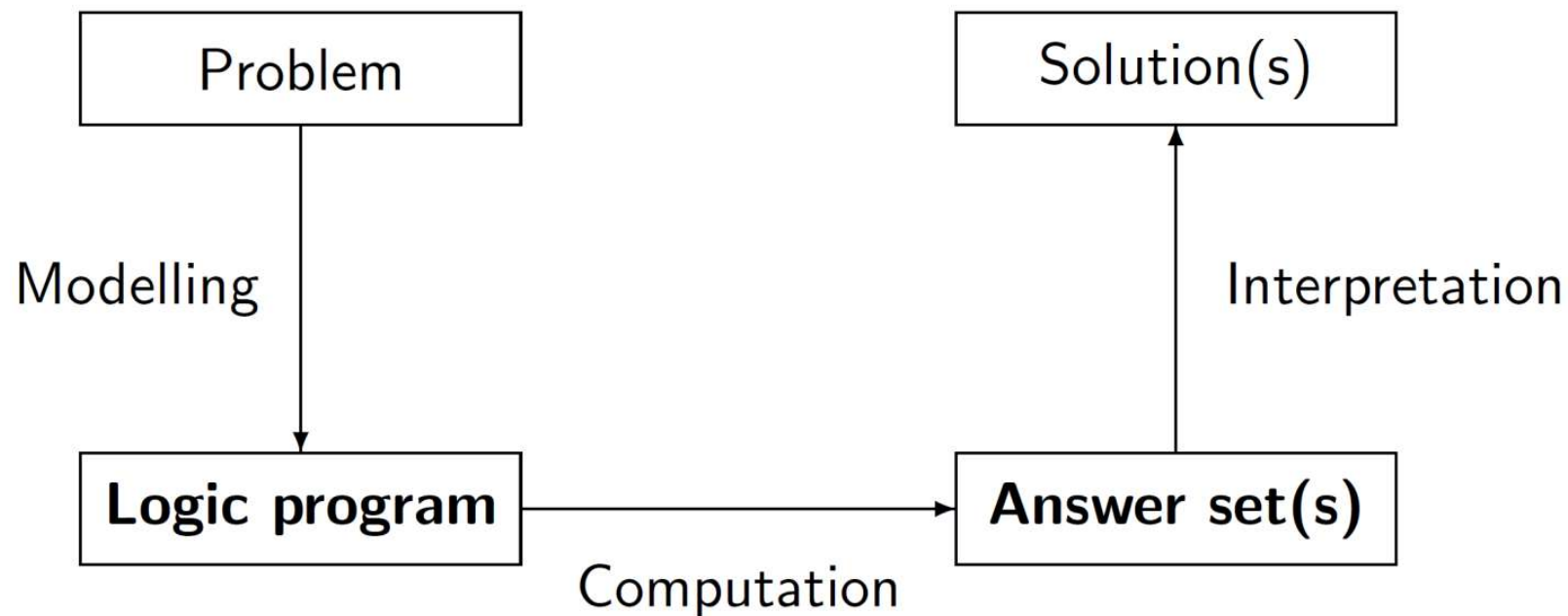
tailored to Knowledge Representation and Reasoning

- ❑ ASP allows for solving all search problems in NP (and NP^{NP}) in a uniform way (being more compact than SAT)
- ❑ Traditional approach (e.g. Prolog):
 - Provide a specification of the problem.
 - A solution is given by a **derivation** of an appropriate query.
- ❑ Model-based approach (e.g. ASP and SAT):
 - Provide a specification of the problem.
 - A solution is given by a **model** of the specification.

ASP as High-level Language

Basic Idea:

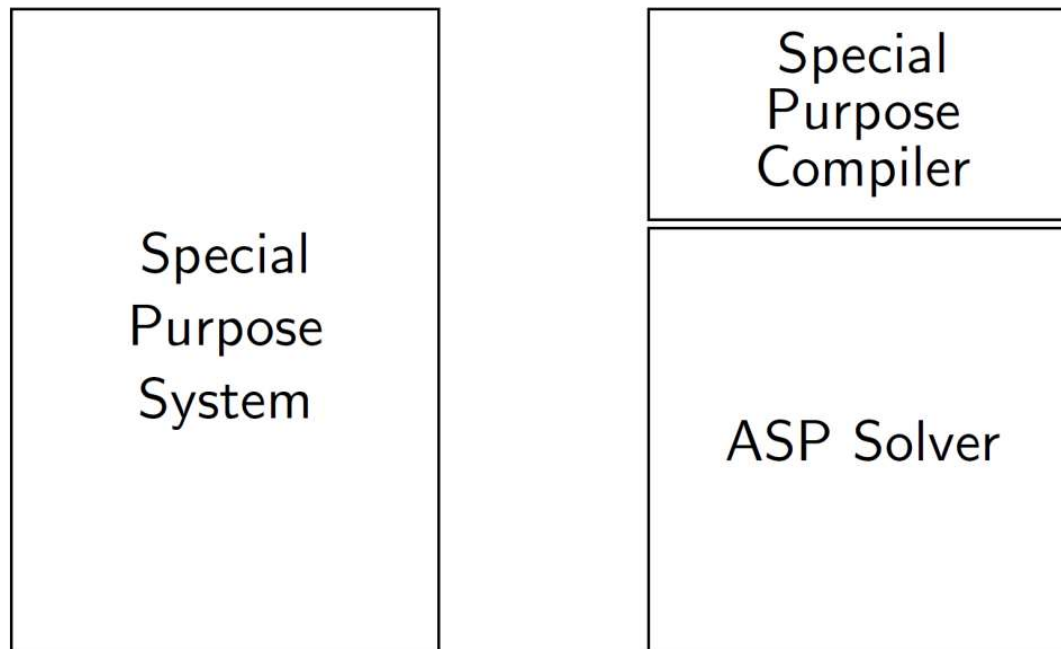
- ❑ Encode problem (class+instance) as a set of rules
- ❑ Read off solutions from answer sets of the rules



ASP as Low-level Language

Basic Idea:

- ❑ Compile a problem automatically into a logic program
- ❑ Solve the original problem by solving its compilation

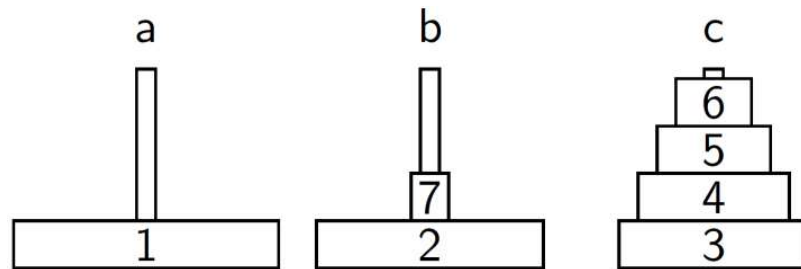


What is ASP good for?

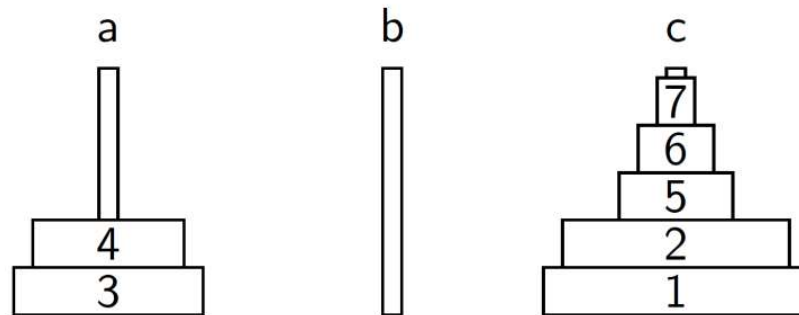
- ❑ Combinatorial search problems (some with substantial amount of data):
 - auctions, bio-informatics, computer-aided verification, configuration, constraint satisfaction, diagnosis, information integration,
 - planning and scheduling, security analysis, semantic web, wire-routing,
 - Zoology
 - ❑ Early 2008 ASP was used as a basis for a decision support system for NASA's space shuttle
 - ❑ ASP offers integration of KR, DB, and search techniques
- $ASP = KR + DB + Search$
- ❑ ASP is compact and has easily maintainable problem representations
 - ❑ It also offers easy handling of dynamic, knowledge intensive applications

An Instance of Tower of Hanoi

Init:



Goal:



```
peg(a;b;c).  
init_on(1,a).  
init_on(2;7,b).  
init_on(3;4;5;6,c).
```

```
disk(1..7).  
goal_on(3;4,a).  
goal_on(1;2;5;6;7,c).  
moves(70).
```

An Encoding of Tower of Hanoi

$\text{on}(D, P, 0) \text{ :- init_on}(D, P).$

$1 \{ \text{move}(D, P, T) : \text{disk}(D) : \text{peg}(P) \} 1 \text{ :- moves}(M), T = 1..M.$

$\text{move}(D, T) \text{ :- move}(D, _, T).$

$\text{on}(D, P, T) \text{ :- move}(D, P, T).$

$\text{on}(D, P, T+1) \text{ :- on}(D, P, T), \text{not move}(D, T+1), \text{not moves}(T).$

$\text{blocked}(D-1, P, T+1) \text{ :- on}(D, P, T), \text{not moves}(T).$

$\text{blocked}(D-1, P, T) \text{ :- blocked}(D, P, T), \text{disk}(D).$

$\text{:- move}(D, P, T), \text{blocked}(D-1, P, T).$

$\text{:- move}(D, T), \text{on}(D, P, T-1), \text{blocked}(D, P, T).$

$\text{:- not } 1 \{ \text{on}(D, P, T) \} 1, \text{disk}(D), \text{moves}(M), T = 1..M.$

$\text{:- goal on}(D, P), \text{not on}(D, P, M), \text{moves}(M).$

Let it run!

```
torsten@raz > gringo toh_instance.lp toh_encoding.lp | clasp --stats
clasp version 1.3.5
Reading from stdin
Solving...
Answer: 1
peg(a) peg(c) peg(b) init_on(1,a) init_on(2,b) ...
move(6,a,1) move(7,a,2) move(5,b,3) move(7,c,4)
move(6,b,5) move(7,b,6) move(4,a,7) move(7,a,8) ...
move(2,c,63) move(7,c,64) move(6,b,65) move(7,b,66)
move(5,c,67) move(7,a,68) move(6,c,69) move(7,c,70)
move(7,70) move(6,69) move(7,68) move(5,67) move(7,66) ...
SATISFIABLE
```

```
Models      : 1+
Time        : 3.280s (Solving: 3.23s 1st Model: 3.23s Unsat: 0.00s)
Choices     : 130907
Conflicts   : 35738
Restarts    : 12
```

Normal logic programs

A (normal) **rule**, r , is an ordered pair of the form

$$L \leftarrow L_1, \dots, L_m, \text{not } L_{m+1}, \dots, \text{not } L_n$$

where $n \geq m \geq 0$, and each A_i ($0 \leq i \leq n$) is an atom.

□ A (normal) **logic program** is a finite set of rules.

□ Notation

$$\begin{aligned} \text{head}(r) &= L \\ \text{body}(r) &= \{L_1, \dots, L_m, \text{not } L_{m+1}, \dots, \text{not } L_n\} \\ \text{body}^+(r) &= \{L_1, \dots, L_m\} \\ \text{body}^-(r) &= \{L_{m+1}, \dots, L_n\} \end{aligned}$$

□ A program is called **positive** if $\text{body}^-(r) = \emptyset$; for all its rules.

Day 10

Answer Set Programming

1. Introduction
- 2. Semantics**
3. Modeling
4. Variation Logic Programs

Answer set: Formal Definition (Positive Programs)

- ❑ A set of atoms X is **closed under** a positive program Π iff for any $r \in \Pi$, $\text{head}(r) \in X$ whenever $\text{body}^+(r) \subseteq X$
 - ✓ X corresponds to a model of Π
- ❑ The **smallest** set of atoms which is closed under a positive program Π is denoted by $Cn(\Pi)$.
 - ✓ $Cn(\Pi)$ corresponds to the $\subseteq$ -smallest model of Π .
- ❑ The set $Cn(\Pi)$ of atoms is the **answer set** of a *positive* program Π .
- ❑ Positive rules are also referred to as **definite clauses** (clauses with exactly one positive atom).
- ❑ Every definite clause is a **Horn clause**, but not vice versa.
- ❑ A Horn clause without a positive atom is called **constraint rule** (written in the form $\leftarrow L_1, \dots, L_m, \text{not } L_{m+1}, \dots, \text{not } L_n$).

(Rough) notational convention

We sometimes use the following notation interchangeably in order to stress the respective view:

	if	and	or	negation as failure	classical negation
source code	$:-$	,		not	$-$
logic program	$\leftarrow$	,	;	<i>not</i> / $\sim$	$\neg$
formula	$\rightarrow$	$\wedge$	$\vee$	$\sim/(\neg)$	$\neg$

Answer set: Basic idea

- Consider the logical formula

$$\Phi = \{q \wedge (q \wedge \neg r \rightarrow p)\}$$

and its three (classical) models:

$$\{p, q\}, \{q, r\}, \{p, q, r\}$$

- Formula Φ has one stable model, called **answer set**: $\{p, q\}$ with the corresponding logic program $\Pi_{\Phi} = \{q \leftarrow, p \leftarrow q, \text{not } r\}$
- Informally, a set X of atoms is an **answer set** of a logic program Π
 - ✓ If X is a (classical) model of Π and
 - ✓ If all atoms in X are **justified** by some rule in Π

Answer Set: Formal Definition (Normal Programs)

□ The reduct, Π^X , of a program Π relative to a set X of atoms is defined by

$$\Pi^X = \{\text{head}(r) \leftarrow \text{body}^+(r) \mid r \in \Pi \text{ and } \text{body}^-(r) \cap X = \emptyset\}.$$

□ A set X of atoms is an answer set of a program Π if $Cn(\Pi^X) = X$.

Recall: $Cn(\Pi^X)$ is the $\subseteq$ -smallest (classical) model of Π^X .

□ Intuition: X is stable under “applying rules from Π ”

□ Note: Every atom in X is justified by an “applying rule from Π ”

□ In other words, given a set X of atoms from Π , Π^X is obtained from Π by deleting

1. each rule having a “*not A*” in its body with $A \in X$ and then
2. all negative atoms of the form “*not A*” in the bodies of the remaining rules.

Examples

$$\square \Pi_1 = \{p \leftarrow p, q \leftarrow \text{not } p\}$$

$$\square \Pi_2 = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$	
$\emptyset$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$	✗
$\{p\}$	$p \leftarrow p$	$\emptyset$	✗
$\{q\}$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$	✓
$\{p, q\}$	$p \leftarrow p$	$\emptyset$	✗

Examples

$$\square \Pi_1 = \{p \leftarrow p, q \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$
$\emptyset$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$ ✗
$\{p\}$	$p \leftarrow p$	$\emptyset$ ✗
$\{q\}$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$ ✓
$\{p, q\}$	$p \leftarrow p$	$\emptyset$ ✗

$$\square \Pi_2 = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$
$\emptyset$	$p \leftarrow$ $q \leftarrow$	$\{p, q\}$ ✗
$\{p\}$	$p \leftarrow$	$\{p\}$ ✓
$\{q\}$	$q \leftarrow$	$\{q\}$ ✓
$\{p, q\}$		$\emptyset$ ✗

$$\square \Pi_3 = \{p \leftarrow \text{not } p\}$$

Examples

$$\square \Pi_1 = \{p \leftarrow p, q \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$	
$\emptyset$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$	✗
$\{p\}$	$p \leftarrow p$	$\emptyset$	✗
$\{q\}$	$p \leftarrow p$ $q \leftarrow$	$\{q\}$	✓
$\{p, q\}$	$p \leftarrow p$	$\emptyset$	✗

$$\square \Pi_3 = \{p \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$	
$\emptyset$	$p \leftarrow$	$\{p\}$	✗
$\{p\}$		$\emptyset$	✗

$$\square \Pi_2 = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$$

X	Π^X	$Cn(\Pi^X)$	
$\emptyset$	$p \leftarrow$ $q \leftarrow$	$\{p, q\}$	✗
$\{p\}$	$p \leftarrow$	$\{p\}$	✓
$\{q\}$	$q \leftarrow$	$\{q\}$	✓
$\{p, q\}$		$\emptyset$	✗

Normal Logic Program Properties

- ❑ A logic program may have zero, one, or multiple answer sets!
- ❑ If X is an answer set of a logic program Π , then X is a model of Π (seen as a formula).
- ❑ If X and Y are answer sets of a normal program Π , then $X \not\subseteq Y$.

Answer set: Alternative Definition

Let Π be a normal program and X a set of atoms.

- The set of generating rules of X relative to Π is defined by
$$\Pi_x = \{r \in \Pi \mid \text{body}^+(r) \subseteq X \text{ and } \text{body}^-(r) \cap X = \emptyset\}$$
- X is an answer set of Π iff X is a $\subseteq$ -minimal model of Π_x .
- Or, X is an answer set of Π iff $X \in \min_{\subseteq}(\Pi_x)$, where $\min_{\subseteq}(\Pi)$ is the set of $\subseteq$ -minimal models of a program Π .
- Example: $\Pi_2 = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$

$$\frac{X}{\emptyset}$$

$$\frac{}{\{p\}}$$

$$\frac{}{\{q\}}$$

$$\frac{}{\{p, q\}}$$

Answer set: Alternative Definition

Let Π be a normal program and X a set of atoms.

- The set of generating rules of X relative to Π is defined by

$$\Pi_x = \{r \in \Pi \mid \text{body}^+(r) \subseteq X \text{ and } \text{body}^-(r) \cap X = \emptyset\}$$
- X is an answer set of Π iff X is a $\subseteq$ -minimal model of Π_x .
- Or, X is an answer set of Π iff $X \in \min_{\subseteq}(\Pi_x)$, where $\min_{\subseteq}(\Pi)$ is the set of $\subseteq$ -minimal models of a program Π .
- Example: $\Pi_2 = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$

X	Π_x	"logically"	$\min_{\subseteq}(\Pi_x)$	
$\emptyset$	$p \leftarrow \text{not } q$ $q \leftarrow \text{not } p$	$p \vee q$	$\{p\}, \{q\}$	✗
$\{p\}$	$p \leftarrow \text{not } q$	$p \vee q$	$\{p\}, \{q\}$	✓
$\{q\}$	$q \leftarrow \text{not } p$	$p \vee q$	$\{p\}, \{q\}$	✓
$\{p, q\}$		$\top$	$\emptyset$	✗

A closer look at Cn

Inductive characterization

Let Π be a positive program and X a set of atoms.

- The **immediate consequence operator** T_Π is defined as follows:

$$T_\Pi = \{\text{head}(r) \mid r \in \Pi \text{ and } \text{body}(r) \subseteq X\}$$

- Iterated applications of T_Π are written as T_Π^j for $j \geq 0$, where $T_\Pi^0 X = X$ and $T_\Pi^i X = T_\Pi T_\Pi^{i-1} X$ for $i \geq 1$.

Theorem

For any positive program Π , we have

- $Cn(\Pi) = \bigcup_{i \geq 0} T_\Pi^i \emptyset$
- $X \subseteq Y$ implies $T_\Pi X \subseteq T_\Pi Y$
- $Cn(\Pi)$ is the smallest fixpoint of T_Π

Example of T_{Π} iteration

$$\Pi = \{p \leftarrow, q \leftarrow, r \leftarrow p, s \leftarrow q, t, t \leftarrow r, u \leftarrow v\}$$

1. $T_{\Pi}^0 \emptyset = \emptyset$
2. $T_{\Pi}^1 \emptyset = \{p, q\} = T_{\Pi} T_{\Pi}^0 \emptyset = T_{\Pi} \emptyset$
3. $T_{\Pi}^2 \emptyset = \{p, q, r\} = T_{\Pi} T_{\Pi}^1 \emptyset = T_{\Pi} \{p, q\}$
4. $T_{\Pi}^3 \emptyset = \{p, q, r, t\} = T_{\Pi} T_{\Pi}^2 \emptyset = T_{\Pi} \{p, q, r\}$
5. $T_{\Pi}^4 \emptyset = \{p, q, r, t, s\} = T_{\Pi} T_{\Pi}^3 \emptyset = T_{\Pi} \{p, q, r, t\}$
6. $T_{\Pi}^5 \emptyset = \{p, q, r, t, s\} = T_{\Pi} T_{\Pi}^4 \emptyset = T_{\Pi} \{p, q, r, t, s\}$
7. $T_{\Pi}^6 \emptyset = \{p, q, r, t, s\} = T_{\Pi} T_{\Pi}^5 \emptyset = T_{\Pi} \{p, q, r, t, s\}$

To see that $Cn(\Pi) = \{p, q, r, t, s\}$ is the smallest fixpoint of T_{Π} , note that $T_{\Pi} \{p, q, r, t, s\} = \{p, q, r, t, s\}$ and $T_{\Pi} X \neq X$ for every $X \subsetneq \{p, q, r, t, s\}$

Day 10

Answer Set Programming

1. Introduction
2. Semantics
- 3. Modeling**
4. Variation Logic Programs

Programs with Variables

Let Π be a logic program.

- ❑ **Herbrand universe** U^Π : Set of constants in Π .
- ❑ **Herbrand base** B^Π : Set of (variable-free) atoms constructible from U^Π
 - ✓ We usually denote this as $\mathcal{A}$ and call it **alphabet**.
- ❑ Ground instances of $r \in \Pi$: Set of variable-free rules obtained by replacing all variables in r by elements from U^Π :

$$ground(r) = \{r\theta \mid \theta: var(r) \rightarrow U^\Pi\}$$

where $var(r)$ stands for the set of all variables occurring in r ; θ is a (ground) substitution.

- ❑ **Ground Instantiation** of Π :

$$ground(\Pi) = \bigcup_{r \in \Pi} ground(r)$$

Let Π be a normal logic program with variables.

We define a set X of (**ground**) atoms as an **answer set** of Π if $Cn(ground(\Pi)^X) = X$.

An Example

□ $\Pi = \{ r(a, b) \leftarrow, r(b, c) \leftarrow, t(X, Y) \leftarrow r(X, Y) \}$

□ $U^\Pi = \{ a, b, c \}$

□ $B^\Pi = \left\{ \begin{array}{l} r(a, a), r(a, b), r(a, c), r(b, a), r(b, b), r(b, c), r(c, a), r(c, b), r(c, c), \\ t(a, a), t(a, b), t(a, c), t(b, a), t(b, b), t(b, c), t(c, a), t(c, b), t(c, c) \end{array} \right\}$

□ $ground(\Pi) = \left\{ \begin{array}{l} r(a, b) \leftarrow, \\ r(b, c) \leftarrow, \\ t(a, a) \leftarrow r(a, a), t(b, a) \leftarrow r(b, a), t(c, a) \leftarrow r(c, a), \\ t(a, b) \leftarrow r(a, b), t(b, b) \leftarrow r(b, b), t(c, b) \leftarrow r(c, b), \\ t(a, c) \leftarrow r(a, c), t(b, c) \leftarrow r(b, c), t(c, c) \leftarrow r(c, c) \end{array} \right\}$

□ **Intelligent Grounding** aims at reducing the ground instantiation.

Language Constructs

□ Variables (over the Herbrand Universe)

- $p(X) \text{ :- } q(X)$ over constants $\{a, b, c\}$ stands for $p(a) \text{ :- } q(a), p(b) \text{ :- } q(b), p(c) \text{ :- } q(c)$

□ Conditional Literals

- $p \text{ :- } \mathbf{q(X) : r(X)}$ given $\mathbf{r(a), r(b), r(c)}$ stands for $p \text{ :- } \mathbf{q(a), q(b), q(c)}$

□ Disjunction

- $\mathbf{p(X) \text{ -- } q(X) \text{ :- } r(X)}$

□ Integrity Constraints

- $\text{:- } q(X), p(X)$

□ Choice

- $\mathbf{2 \{ p(X,Y) : q(X) \} \text{ -- } r(Y)}$

□ Aggregates

- $s(Y) \text{ :- } r(Y), \mathbf{2 \#count \{ p(X,Y) : q(X) \} \text{ -- } 7}$

- also: $\#sum, \#avg, \#min, \#max, \#even, \#odd$

Day 10

Answer Set Programming

1. Introduction
2. Semantics
3. Modeling
4. **Variation Logic Programs**

Disjunctive logic programs

- A **disjunctive rule**, r , is an ordered pair of the form

$$A_1; \dots; A_m \leftarrow A_{m+1}, \dots, A_n, \text{not } A_{n+1}, \dots, \text{not } A_o,$$

where $o \geq n \geq m \geq 0$, and each A_i ($0 \leq i \leq o$) is an atom.

- A **disjunctive logic program** is a finite set of disjunctive rules.

- (Generalized) Notation

$$\begin{aligned} \text{head}(r) &= \{A_1, \dots, A_m\} \\ \text{body}(r) &= \{A_{m+1}, \dots, A_n, \text{not } A_{n+1}, \dots, \text{not } A_o\} \\ \text{body}^+(r) &= \{A_{m+1}, \dots, A_n\} \\ \text{body}^-(r) &= \{A_{n+1}, \dots, A_o\} \end{aligned}$$

- A program is called **positive** if $\text{body}^-(r) = \emptyset$ for all its rules.

Answer sets (Disjunctive LP)

Positive programs:

- A set of atoms X is **closed under** a positive program Π iff for any $r \in \Pi$, $\text{head}(r) \cap X \neq \emptyset$ whenever $\text{body}^+(r) \subseteq X$
 - ✓ X corresponds to a model of Π .
- The set of all $\subseteq$ -minimal sets of atoms being closed under a positive program Π is denoted by $\text{min}_{\subseteq}(\Pi)$.
 - ✓ $\text{min}_{\subseteq}(\Pi)$ corresponds to the $\subseteq$ -minimal models of Π .

Disjunctive programs:

- The **reduct**, Π^X , of a disjunctive program Π relative to a set X of atoms is defined by
$$\Pi^X = \{\text{head}(r) \leftarrow \text{body}^+(r) \mid r \in \Pi \text{ and } \text{body}^-(r) \cap X = \emptyset\}$$
- A set X of atoms is an **answer set** of a disjunctive program Π if $X \in \text{min}_{\subseteq}(\Pi^X)$.

Disjunctive LP Examples

- Example 1: $\Pi = \left\{ \begin{array}{l} a \leftarrow, \\ b; c \leftarrow a \end{array} \right\}$
 - ✓ The sets $\{a, b\}$, $\{a, c\}$, and $\{a, b, c\}$ are closed under Π .
 - ✓ We have $\min_{\subseteq}(\Pi) = \{\{a, b\}, \{a, c\}\}$.
- Example 2: $\Pi_1 = \{a; b; c \leftarrow\}$ has an answer set $\{a\}$, $\{b\}$, and $\{c\}$.
- Example 3: $\Pi_2 = \{a; b; c \leftarrow, \leftarrow a\}$ has an answer set $\{b\}$, and $\{c\}$.
- Example 4: $\Pi_3 = \{a; b; c \leftarrow, \leftarrow a, b \leftarrow c, c \leftarrow b\}$ has an answer set $\{b, c\}$.
- Example 5: $\Pi_4 = \{a; b \leftarrow c, b \leftarrow \text{not } a, \text{not } c, a; c \leftarrow \text{not } b\}$ has an answer set $\{a\}$ and $\{b\}$.

Disjunctive LP Properties

- A disjunctive logic program may have zero, one, or multiple answer sets.
- If X is an answer set of a disjunctive logic program Π then X is a model of Π .
- If X and Y are answer sets of a disjunctive logic program Π , $X \not\subseteq Y$.
- If $A \in X$ for some answer X set of a disjunctive logic program Π , then there is a rule $r \in \Pi_X$ such that $\{A\} = \text{head}(r) \cap X$.

□ Example: $\Pi = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(X); c(Y) \leftarrow a(X,Y), \text{not } c(Y) \end{array} \right\}$

□ $\text{ground}(\Pi) = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(1); c(1) \leftarrow a(1,1), \text{not } c(1) \\ b(1); c(2) \leftarrow a(1,2), \text{not } c(2) \\ b(2); c(1) \leftarrow a(2,1), \text{not } c(1) \\ b(2); c(2) \leftarrow a(2,2), \text{not } c(2) \end{array} \right\}$

For every answer set X of Π , we have:

- $a(1,2) \in X$ and
- $\{a(1,1), a(2,1), a(2,2)\} \cap X = \emptyset$

Disjunctive LP (Another) Example

- Example: $\Pi = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(X); c(Y) \leftarrow a(X,Y), \text{not } c(Y) \end{array} \right\}$
- $\text{ground}(\Pi)^X = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(1); c(1) \leftarrow a(1,1), \text{not } c(1) \\ b(1); c(2) \leftarrow a(1,2), \text{not } c(2) \\ b(2); c(1) \leftarrow a(2,1), \text{not } c(1) \\ b(2); c(2) \leftarrow a(2,2), \text{not } c(2) \end{array} \right\}$
- Consider $X = \{X(1,2), b(1)\}$
- We get $\min_{\subseteq}(\text{ground}(\Pi)^X) = \{\{a(1,2), b(1)\}, \{a(1,2), c(2)\}\}$.
- X is an answer set of Π because $X \in \min_{\subseteq}(\text{ground}(\Pi)^X)$.

Disjunctive LP (Another) Example

- Example: $\Pi = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(X); c(Y) \leftarrow a(X,Y), \text{not } c(Y) \end{array} \right\}$
- $\text{ground}(\Pi)^X = \left\{ \begin{array}{l} a(1,2) \leftarrow, \\ b(1); c(1) \leftarrow a(1,1), \text{not } c(1) \\ b(1); c(2) \leftarrow a(1,2), \text{not } c(2) \\ b(2); c(1) \leftarrow a(2,1), \text{not } c(1) \\ b(2); c(2) \leftarrow a(2,2), \text{not } c(2) \end{array} \right\}$
- Consider $X = \{X(1,2), c(2)\}$
- We get $\min_{\subseteq}(\text{ground}(\Pi)^X) = \{\{a(1,2)\}\}$.
- X is **no** answer set of Π because $X \notin \min_{\subseteq}(\text{ground}(\Pi)^X)$.

Nested Logic Programs

- ❑ Formulas are formed from
 - propositional atoms and
 - $\perp$ and $\top$
- Using
 - negation-as-failure (not),
 - conjunction (,), and
 - disjunction (;).
- ❑ A **nested rule**, r , is an ordered pair of the form $F \leftarrow G$ where F and G are formulas.
- ❑ A **nested program** is a finite set of rules.
- ❑ Notation: $head(r) = F$ and $body(r) = G$.

Nested Logic Programs

The **satisfaction relation** $X \vdash F$ between a set of atoms and a formula F is defined recursively as follows:

- $X \vdash F$ if $F \in X$ for an atom F ,
- $X \vdash \textcolor{red}{T}$
- $X \not\vdash \textcolor{red}{\perp}$
- $X \vdash (F, G)$ if $X \vdash F$ and $X \vdash G$,
- $X \vdash (F; G)$ if $X \vdash F$ or $X \vdash G$,
- $X \vdash \textit{not } F$ if $X \not\vdash F$.

A set X of atoms satisfies a nested program Π , written $X \vdash \Pi$, iff for any $r \in \Pi$, $X \vdash \textit{head}(r)$ whenever $X \vdash \textit{body}(r)$.

The set of all $\subseteq$ -minimal sets of atoms satisfying program Π is denoted by $\min_{\subseteq}(\Pi)$

Nested Logic Programs

The **reduct**, F^X , of a formula F relative to a set X of atoms is defined recursively as follows:

- $F^X = F$ if F is an atom or $\perp$ or $\top$,
- $(F, G)^X = (F^X, G^X)$,
- $(F; G)^X = (F^X; G^X)$,
- $(\text{not } F)^X = \begin{cases} \perp & \text{if } X \vdash F \\ \top & \text{otherwise} \end{cases}$
- The **reduct**, Π^X , of a nested program Π relative to a set X of atoms is defined by $\Pi^X = \{head(r)^X \leftarrow body(r)^X \mid r \in \Pi\}$
- A set X of atoms is an **answer set** of a nested program Π if $X \in \min_{\subseteq}(\Pi^X)$.

Nested Program Examples

□ $\Pi_1 = \{(p; \text{not } p) \leftarrow \textcolor{red}{T}\}$

□ For $X = \emptyset$, we get

- $\Pi_1^\emptyset = \{(p; \textcolor{red}{T}) \leftarrow \textcolor{red}{T}\}$
- $\min_{\subseteq}(\Pi_1^\emptyset) = \{\emptyset\}$.

□ For $X = \{p\}$, we get

- $\Pi_1^{\{p\}} = \{(p; \textcolor{red}{\perp}) \leftarrow \textcolor{red}{T}\}$
- $\min_{\subseteq}(\Pi_1^{\{p\}}) = \{p\}$.

□ $\Pi_2 = \{p \leftarrow \text{not not } p\}$

□ For $X = \emptyset$, we get $\Pi_2^\emptyset = \{p \leftarrow \textcolor{red}{\perp}\}$ and $\min_{\subseteq}(\Pi_2^\emptyset) = \{\emptyset\}$.

□ For $X = \{p\}$, we get $\Pi_2^{\{p\}} = \{p \leftarrow \textcolor{red}{T}\}$ and $\min_{\subseteq}(\Pi_2^{\{p\}}) = \{\{p\}\}$.

Nested Program Examples

□ In general

- $F \leftarrow G, \text{not not } H$ is equivalent to $F; \text{not } H \leftarrow G$
- $F; \text{not not } G \leftarrow H$ is equivalent to $F \leftarrow H, \text{not } G$

Propositional Theories

□ Formulas are formed from

- propositional atoms and

- $\perp$

Using

- implication ($\rightarrow$),

- conjunction ($,$), and

- disjunction ($;$).

□ Notation:

□ $\top = (\perp \rightarrow \perp)$

□ $\sim F = (F \rightarrow \perp)$ (or: not F)

□ A **propositional theory** is a finite set of formulas.

Propositional Theories

- The **satisfaction relation** $X \vdash F$ between a set X of atoms and a (set of) formula(s) F is defined as in propositional logic.
- The **reduct**, F^X , of a formula F relative to a set X of atoms is defined recursively as follows:
 - $F^X = \perp$ if $X \not\vdash F$,
 - $F^X = F$ if $F \in X$
 - $F^X = (G^X \circ H^X)$ if $X \vdash F$ and $F = (G \circ H)$ for $\circ \in \{\wedge, \vee, \rightarrow\}$
 - If $F = \sim G = (G \rightarrow \perp)$, then $F^X = (\perp \rightarrow \perp) = \top$ if $X \not\vdash G$, and $F^X = \perp$ otherwise
- The **reduct**, $\mathcal{F}^X$, of a propositional theory $\mathcal{F}$ relative to a set X of atoms is defined as
$$\mathcal{F}^X = \{F^X \mid F \in \mathcal{F}\}.$$
- The set of all $\subseteq$ -minimal sets of atoms satisfying a propositional theory $\mathcal{F}$ is denoted by $\min_{\subseteq}(\mathcal{F})$.
- A set X of atoms is an **answer set** of a propositional theory $\mathcal{F}$ if $X \in \min_{\subseteq}(\mathcal{F}^X)$.

Propositional Theory Examples

□ If X is an answer set of $\mathcal{F}$, then

- $X \vdash \mathcal{F}$ and
- $\min_{\subseteq}(\mathcal{F}^X) = \{X\}$;
- In general, this does not imply $X \in \min_{\subseteq}(\mathcal{F})$!

Let's check if X is an answer set for propositional theory $\mathcal{F}$

□ $\mathcal{F}_1 = \{p \vee (p \rightarrow (q \wedge r))\}$

- For $X = \{p, q, r\}$, we get

$$\mathcal{F}_1^{\{p,q,r\}} = \{p \vee (p \rightarrow (q \wedge r))\} \text{ and } \min_{\subseteq}(\mathcal{F}_1^{\{p,q,r\}}) = \{\emptyset\}$$

WRONG!

- For $X = \emptyset$, we get

$$\mathcal{F}_1^{\emptyset} = \{\perp \vee (\perp \rightarrow \perp)\} \text{ and } \min_{\subseteq}(\mathcal{F}_1^{\emptyset}) = \{\emptyset\}$$

CORRECT!

Propositional Theory Examples

□ $\mathcal{F}_2 = \{p \vee (\sim p \rightarrow (q \wedge r))\}$

- For $X = \emptyset$, we get

$$\mathcal{F}_2^\emptyset = \{\perp\} \text{ and } \min_{\subseteq}(\mathcal{F}_1^\emptyset) = \emptyset.$$

WRONG!

- For $X = \{p\}$, we get

$$\mathcal{F}_2^{\{p\}} = \{p \vee (\perp \rightarrow \perp)\} \text{ and } \min_{\subseteq}(\mathcal{F}_2^{\{p\}}) = \{\emptyset\}.$$

WRONG!

- For $X = \{q, r\}$, we get

$$\mathcal{F}_2^{\{q, r\}} = \{\perp \vee (\top \rightarrow (q \wedge r))\} \text{ and } \min_{\subseteq}(\mathcal{F}_2^{\{q, r\}}) = \{\{q, r\}\}.$$

CORRECT!

Relationship with Logic Programs

□ The translation, $\tau[(F \leftarrow G)]$, of a (nested) rule $(F \leftarrow G)$ is defined recursively as follows:

- ❖ $\tau[(F \leftarrow G)] = (\tau[G] \rightarrow \tau[F])$,
- ❖ $\tau[\perp] = \perp$,
- ❖ $\tau[\top] = \top$,
- ❖ $\tau[F] = F$ if F is an atom,
- ❖ $\tau[\text{not } F] = \sim\tau[F]$,
- ❖ $\tau[F, G] = (\tau[F] \wedge \tau[G])$,
- ❖ $\tau[F; G] = (\tau[F] \vee \tau[G])$.

□ The translation of a logic program Π is $\tau[\Pi] = \{\tau[r] \mid r \in \Pi\}$.

□ Given a logic program Π and a set X of atoms, X is **an answer set** of Π iff X is **an answer set** of $\tau[\Pi]$.

Logic Programs as propositional theories

□ The normal logic program $\Pi = \{p \leftarrow \text{not } q, q \leftarrow \text{not } p\}$

corresponds to $\tau[\Pi] = \{\sim q \rightarrow p, \sim p \rightarrow q\}$

- Answer sets: $\{p\}$ and $\{q\}$

□ The disjunctive logic program $\Pi = \{p; q \leftarrow\}$

corresponds to $\tau[\Pi] = \{\textcolor{red}{T} \rightarrow p \vee q\}$

- Answer sets: $\{p\}$ and $\{q\}$

□ The nested logic program $\Pi = \{p \leftarrow \text{not not } p\}$

corresponds to $\tau[\Pi] = \{\sim\sim p \rightarrow p\}$

- Answer sets: $\{p\}$ and $\emptyset$

Complexity

Let A be an atom and X be a set of atoms.

- ❑ For a **positive normal logic** program Π :
 - ❑ Deciding whether X is the answer set of Π is **P-complete**.
 - ❑ Deciding whether A is in the answer set of Π is **P-complete**.
- ❑ For a **normal logic program** Π :
 - ❑ Deciding whether X is an answer set of Π is **P-complete**.
 - ❑ Deciding whether A is in an answer set of Π is **NP-complete**.

Complexity

Let A be an atom and X be a set of atoms.

- ❑ For a **positive disjunctive logic program** Π :
 - ❑ Deciding whether X is the answer set of Π is **co-NP-complete**.
 - ❑ Deciding whether A is in the answer set of Π is **NP^{NP}-complete**.
- ❑ For a **disjunctive logic program** Π :
 - ❑ Deciding whether X is an answer set of Π is **co-NP-complete**.
 - ❑ Deciding whether A is in the answer set of Π is **NP^{NP}-complete**.
- ❑ For a **nested logic program** Π :
 - ❑ Deciding whether X is the answer set of Π is **co-NP-complete**.
 - ❑ Deciding whether A is in the answer set of Π is **NP^{NP}-complete**.
- ❑ For a **propositional theory** $\mathcal{F}$:
 - ❑ Deciding whether X is an answer set of Π is **co-NP-complete**.
 - ❑ Deciding whether A is in the answer set of Π is **NP^{NP}-complete**.

Summary

- ASP provides a specification of the problem where a solution is given by a model of the specification.
- Answer set is a stable and minimal model that provides an entailment to a logic program
- There are variations of logic programs that can be translated into propositional theories.